

Требования к проекту перед модулем по масштабированию баз данных

1. Общая идея

До начала модуля у каждого студента должен быть готов собственный backend-сервис.

На занятиях мы не будем разрабатывать CRUD-сервис с нуля. Вместо этого будем использовать уже существующий проект как основу и постепенно масштабировать его базу данных.

В течение модуля ваш сервис будет проходить несколько этапов:

Обычная PostgreSQL
↓
Оптимизация запросов
↓
Индексы
↓
Партиционирование
↓
Primary + Replica
↓
Масштабирование чтения
↓
Шардирование

Поэтому к следующему занятию сервис должен быть полностью работоспособен.

2. Предметная область

Каждый студент самостоятельно выбирает предметную область для своего сервиса.

Примеры:

- интернет-магазин;
- библиотека;
- система бронирования;
- кинотеатр;
- музыкальный сервис;
- социальная сеть;
- система управления мероприятиями;
- доставка;
- образовательная платформа;

- трекер задач;
- игровая платформа.

Тема должна предполагать работу с относительно большим количеством данных.

Не рекомендуется выбирать сервис, состоящий из одной простой таблицы.

3. Технологический стек

Язык программирования и backend-фреймворк можно выбирать самостоятельно.

Например:

- Java / Spring Boot;
- Kotlin / Ktor или Spring;
- Go;
- Python / FastAPI, Django и т.д.;
- Node.js / NestJS, Express и т.д.;
- C# / ASP.NET Core;
- PHP / Laravel;
- другой согласованный стек.

Обязательные технологии

- PostgreSQL;
- Docker;
- Docker Compose;
- система миграций БД.

ORM использовать можно, но SQL-запросы должны быть вам понятны и доступны для анализа.

4. Архитектура

Сервис должен иметь как минимум следующую структуру:

```
Client
  ↓
Backend API
  ↓
PostgreSQL
```

Контроллеры/handlers не должны напрямую работать с базой данных.

Должно существовать отдельное место для работы с БД:

```
Controller / Handler
    ↓
Service
    ↓
Repository / Data Access
    ↓
PostgreSQL
```

Конкретная реализация зависит от выбранного языка.

5. База данных

В проекте должна использоваться PostgreSQL.

База данных должна запускаться через Docker Compose.

Например:

```
docker compose up
```

должен запускать необходимую инфраструктуру проекта.

Схема базы данных не должна создаваться вручную через GUI.

Все изменения структуры БД должны выполняться через систему миграций.

6. Требования к структуре БД

В базе должно быть **не менее 4 связанных таблиц**.

Желательно, чтобы модель содержала:

- primary keys;
 - foreign keys;
 - one-to-many связи;
 - хотя бы одну many-to-many связь.
-

7. Основная сущность для масштабирования

В проекте должна быть хотя бы одна сущность, количество записей которой естественным образом может значительно увеличиваться.

Например:

- orders;
- transactions;
- messages;
- posts;
- events;
- logs;
- bookings;
- comments;
- payments.

Эту сущность в дальнейшем мы будем использовать для экспериментов с производительностью, партиционированием и шардированием.

В README необходимо указать:

Основная сущность для масштабирования:
<название таблицы>

Почему она подходит:
<краткое объяснение>

8. CRUD API

Для основных сущностей необходимо реализовать CRUD.

Минимально:

```
GET    /api/<entities>
GET    /api/<entities>/<id>
POST   /api/<entities>
PUT    /api/<entities>/<id>
DELETE /api/<entities>/<id>
```

Не обязательно реализовывать полный CRUD абсолютно для каждой технической таблицы.

9. Связи между сущностями

API должен позволять работать со связанными сущностями.

Например:

```
GET /users/{id}/orders
GET /orders/{id}/items
GET /products/{id}/reviews
```

Нам понадобится это для последующего анализа JOIN-запросов.

10. JOIN-запросы

В сервисе должно быть минимум **2 нетривиальных SQL-запроса с JOIN**.

Например:

```
SELECT ...
FROM orders
JOIN users ON ...
JOIN order_items ON ...
JOIN products ON ...
WHERE ...
```

Запросы должны использоваться сервисом, а не существовать только в отдельном .sql файле.

В README необходимо указать минимум два таких запроса.

11. Агрегирующие запросы

Должен быть минимум **1 запрос с агрегацией**.

Например:

```
COUNT
SUM
AVG
MIN
MAX
GROUP BY
```

Примеры:

- Количество заказов пользователя
- Продажи по категориям
- Средняя стоимость заказа
- Количество сообщений за период
- Количество бронирований по событиям

12. Pagination и Filtering

Хотя бы один endpoint должен поддерживать pagination.

Например:

```
GET /api/orders?page=1&pageSize=20
```

И хотя бы один фильтр:

```
GET /api/orders?status=PAID
```

```
GET /api/orders?from=2026-01-01&to=2026-02-01
```

```
GET /api/products?search=phone
```

Желательно также поддерживать сортировку:

```
?sort=created_at
```

```
?sort=-created_at
```

13. Временное поле

Для основной сущности должна быть колонка, отражающая время создания или изменения записи.

Например:

```
created_at
```

```
updated_at
```

Рекомендуется иметь created_at у основной растущей таблицы.

Это понадобится для работы с партиционированием.

14. Реальные SQL-запросы

В проекте должны присутствовать запросы, которые потенциально могут стать интересными при увеличении объема данных.

Например:

Поиск

```
WHERE name ILIKE ...
```

Фильтрация

```
WHERE status = ...
```

Диапазон

WHERE created_at **BETWEEN** ... **AND** ...

JOIN

JOIN ...

Сортировка

ORDER BY created_at **DESC**

Агрегация

GROUP BY ...

Не нужно специально делать их медленными.

На модуле мы сами увеличим объем данных и посмотрим, какие запросы станут проблемными.

15. Тестовые данные

К началу модуля сервис должен иметь небольшой набор тестовых данных, достаточный для проверки API.

Кроме того, необходимо предусмотреть возможность массовой генерации данных.

Например:

100 000 пользователей
1 000 000 заказов
5 000 000 событий

Конкретные объемы будут определены на занятиях.

Генератор данных можно написать самостоятельно или использовать подходящий инструмент/скрипт.

Важно, чтобы данные можно было генерировать автоматически, а не добавлять вручную через API.

16. Docker Compose

Проект должен запускаться через Docker Compose.

Минимально:

backend
postgres

Если в проекте уже используются дополнительные компоненты, их также можно оставить.

Главное требование:

`docker compose up`

должен запускать рабочее приложение.

17. Health Check

Необходимо иметь endpoint:

`GET /health`

Он должен позволять понять, что приложение работает и может подключиться к PostgreSQL.

18. Документация API

API должен иметь документацию.

Предпочтительно использовать:

- Swagger / OpenAPI;
- либо аналогичный инструмент для выбранного языка.

В документации должны быть описаны основные endpoint'ы, параметры и тела запросов.

19. README

В корне проекта должен находиться README.md.

Он должен содержать:

О проекте

Краткое описание предметной области.

Запуск

Например:

`docker compose up`

Архитектура

Краткое описание компонентов.

Схема БД

Диаграмма или описание таблиц.

Основные endpoint'ы

Список API.

Основная сущность для масштабирования

orders

и объяснение, почему именно она.

Сложные запросы

Минимум:

- 2 JOIN-запроса;
- 1 агрегирующий запрос.

Генерация данных

Описание того, как получить большое количество тестовых данных.

20. Что НЕ требуется до начала модуля

До начала занятий **не нужно** самостоятельно реализовывать:

- репликацию;
- партиционирование;
- шардирование;
- consistent hashing;
- shard router;
- распределенные транзакции;
- автоматический failover.

Это будет предметом модуля.

Также не нужно заранее оптимизировать все запросы.

Наоборот, нам важно получить исходное состояние системы и самостоятельно измерить, как она ведет себя при росте нагрузки.

21. Что должно быть готово к следующему занятию

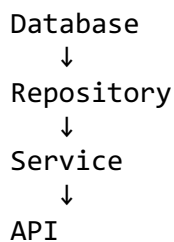
К началу первой практики ваш проект должен удовлетворять следующему чек-листу:

- ☐ Backend-сервис запускается
 - ☐ PostgreSQL запускается через Docker
 - ☐ Миграции применяются автоматически
 - ☐ В БД минимум 4 связанные таблицы
 - ☐ Есть CRUD для основных сущностей
 - ☐ Есть связи между сущностями
 - ☐ Есть минимум 2 JOIN-запроса
 - ☐ Есть минимум 1 агрегирующий запрос
 - ☐ Есть pagination
 - ☐ Есть filtering
 - ☐ Есть поле `created_at` у основной растущей сущности
 - ☐ Есть генерация большого объема тестовых данных или возможность ее быстро добавить
 - ☐ Есть `/health`
 - ☐ Есть API documentation
 - ☐ Есть README
 - ☐ Проект запускается одной командой через Docker Compose
-

22. Важное требование

Сервис должен быть вашим собственным проектом.

На модуле мы будем изменять не только SQL, но и архитектуру приложения:



Поэтому необходимо понимать собственный код и уметь найти место, где приложение устанавливает соединение с PostgreSQL и выполняет запросы.
